claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_98c1f280-b9f\agent-a737f8c64d50029bd.jsonl`
- SHA-256: `437ab580b52070b56a20d4840a4bf37cbb3bf410d7f7bc9181086be4840c1679`
- Source modified: `2026-06-14T06:34:32+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_98c1f280-b9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T06:32:15.327Z)

In the current badminton-highlight-indexer repo (cwd), audit ONE signal/component. Verify and return:
- present: the code exists.
- default_off: it does NOT run in the served/default path by default (e.g. cue_flags default False, default_segmenter is NOT 'fusion', the feature is not weighted by the shipped scorer unless explicitly enabled).
- no_special_casing: the signal enters the DECISION as a confidence-scored / feature COLUMN the learned scorer weights ? NOT a hard if/else branch hard-coded into the decision path.
- retained: NOTHING deletes or disables the code because the n=6 measured ?F1 was non-significant; the producer + feature math still exist and are re-measurable when the corpus grows.
Give evidence as concrete file:line refs with short quotes. List concerns. Be precise and skeptical; read the actual files.
COMPONENT: the SCORING seam / no-special-casing. Read backend/eval/experiment.py (Variant.feature_names, compare, the learned scorer e.g. LogisticRegression). Confirm cues enter ONLY as extra feature_names columns the learned scorer weights ? adding/removing a cue is a feature-list change, never an if/else in the decision path. This is the crux of "no special-casing".

2. assistant (2026-06-14T06:32:17.604Z)

I'll audit the SCORING seam / no-special-casing component. Let me start by reading the key file.

3. user (2026-06-14T06:32:18.473Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17      (per-frame signals ? run once on the GPU/WSL box, persisted into
18      `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19      signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
```

```
    given persisted candidate features it produces `List[Interval]` deterministically,
    with no GPU and zero production risk. So most experiments are pure offline
    re-scoring of stored data and never touch the served pipeline.

    Three modes (`EXPERIMENT_HARNESS.md` §5):
      - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
        aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
        held-out loop, but variant-parameterised.
      - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
        variants agree / diverge (A-only, B-only, agreed). The divergences are what a
        human spot-checks (and what two highlight reels would show side by side).
      - the promotion gate stays `run_regression.py` + `regression_baseline.json`
        (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

    Pure + offline + unit-tested. The only DB-touching helper is
    `load_video_candidates`, which reads persisted candidates via
    `Database.query_candidate_segments`.
    """
    from __future__ import annotations

    import json
    import logging
    import os
    from dataclasses import dataclass, field
    from datetime import datetime, timezone
    from hashlib import sha1
    from typing import Any, Callable, Dict, List, Optional, Sequence

    from backend.eval.calibration import pct_improvement
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval.metrics import Interval, match_intervals, score
    from backend.eval.regression import gt_hash
    from backend.eval.training import label_candidates
    from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
    from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
    from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

    logger = logging.getLogger(__name__)

    # Where a comparison run appends one reproducible record. Tracked location (NOT under
    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
    # regression.DEFAULT_BASELINE_PATH.
    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
```

```
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

```

```
        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
```

```
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
```

```
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                       # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
```

```
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                       # honest=False ? one model over all
videos
                model = all_model
            else:                                   # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None               # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))
```

```
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                              "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
```

```
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]

        # One fingerprint over the whole label set ? detects "the deltas were measured
against
        # different labels" the same way regression.gt_hash guards the no-regression
baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ------------------------------------------------------------- SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
```

```
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's
``stats``
        JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
```

```
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

4. user (2026-06-14T06:32:20.754Z)

```
Already up to date.
```

5. assistant (2026-06-14T06:32:23.456Z)

```
Now let me read the supporting files to verify the claims: the classifier (LogisticRegression),
the gate logic in production, and the served default path.
```

6. user (2026-06-14T06:32:24.715Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
20
21      class LogisticRegression:
22          def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23              self.lr = lr
24              self.epochs = epochs
25              self.l2 = l2
26              self.w: Optional[np.ndarray] = None
27              self.b: float = 0.0
28              self.mean: Optional[np.ndarray] = None
29              self.std: Optional[np.ndarray] = None
30              self.feature_names: Optional[List[str]] = None
31
32          @staticmethod
33          def _sigmoid(z: np.ndarray) -> np.ndarray:
34              # Clip for numerical stability (avoid overflow in exp).
```

```
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "LogisticRegression":
            m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
```

```
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101             m.b = float(d.get("b", 0.0))
102             m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103             m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104             m.feature_names = d.get("feature_names")
105             return m
106
107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115
```

7. user (2026-06-14T06:32:25.407Z)

```
Found 38 files
docs\QUALITY_ITERATIONS.md
tests\test_experiment_honest.py
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\EXPERIMENT_HARNESS.md
backend\eval\experiment.py
backend\eval\fusion_golden.py
docs\ANALYZERS_AND_RECONCILERS.md
backend\config\models.py
run_process_and_stitch.py
tests\test_config.py
tests\test_rally_gate.py
tests\test_fusion_hybrid.py
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters\fusion_hybrid.py
tests\test_experiment.py
docs\HOW_RALLY_DETECTION_WORKS.md
CLAUDE.md
config.json
README.md
docs\COMMERCIALIZATION.md
backend\main.py
backend\eval\calibrate_local.py
backend\eval\distill_local.py
backend\pipeline\detectors\__init__.py
backend\eval\eval_partial_labels.py
backend\reporting\harvest.py
backend\reporting\spec.yaml
tests\test_reporting_harvest.py
docs\TRACKNET_WSL_SETUP.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
backend\eval\export_wasb_segments.py
tests\test_calibrate_local.py
tests\test_export_wasb_segments.py
```

8. assistant (2026-06-14T06:32:31.553Z)

Now let me verify the served/default path. I need to check the default segmenter in production config and how cue_flags / min_crossings interact with the served pipeline. Let me look at the config models and the fusion_hybrid segmenter.

9. user (2026-06-14T06:32:32.530Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
```

```
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73          keep_frames: bool = False
74          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75          min_free_gb: float = 0.0
76
77
78      class FusionConfig(BaseModel):
79          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81          Every default reproduces control behaviour: the fusion segmenter persists the
82          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83          true ? the person-cue features, but it does NOT gate the served handoff unless a
84          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86          supplied ? off-court persons (spectators / adjacent courts) are excluded.
87          """
88          # Which trajectory substrate seeds the windows (shuttle stays primary).
89          substrate: Literal["wasb", "tracknet"] = "wasb"
90          # Windowing velocity threshold for the fusion family (the engine reads
91          # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92          # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93          velocity_threshold: float = 0.02
94          # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95          # enabled ? so the default path never imports torch / touches the GPU.
96          cue_flags: dict[str, bool] = Field(default_factory=dict)
97          # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98          # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99          min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08
115         min_segment_duration: float = 3.0
116         dead_time_merge_gap: float = 2.0
```

```
        chunk_duration: float = 90.0
        chunk_overlap: float = 10.0
        detector_model: str = "fasterrcnn_resnet50_fpn"
        detector_score_threshold: float = 0.5
        yolo_velocity_threshold: float = 0.15
        yolo_frame_skip: int = 3
        yolo_tracker: str = "bytetrack.yaml"
        yolo_prefetch_workers: int = 2
        min_action_window_duration: float = 2.0
        log_yolo_candidates: bool = True
        store_yolo_candidates: bool = True
        ai_handoff_padding: float = 2.0
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
```

```
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]


    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
```

```
239         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
240         security: SecurityConfig = Field(default_factory=SecurityConfig)
241         logging: LoggingConfig = Field(default_factory=LoggingConfig)
242 
243         # Allow extra keys during transition so existing config.json files do not break.
244         model_config = {
245             "extra": "allow",
246             "validate_assignment": True,
247         }
248 
249         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
250         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
251         # attribute access / the typed config. A nested *section* is returned as a plain
dict
252         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
253         # section model instead breaks it (a model has no .get), which bit every validator
on
254         # real runs. Scalar leaves are returned as-is.
255         def get(self, key: str, default: Any = None) -> Any:
256             _MISSING = object()
257             val: Any = _MISSING
258             if key in type(self).model_fields or hasattr(self, key):
259                 val = getattr(self, key, _MISSING)
260             if val is _MISSING:
261                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
262                 # discovered dynamically from the model fields, so a future top-level
section
263                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
264                 for fname in type(self).model_fields:
265                     section = getattr(self, fname, None)
266                     if isinstance(section, BaseModel) and hasattr(section, key):
267                         val = getattr(section, key)
268                         break
269             if val is _MISSING:
270                 return default
271             if isinstance(val, BaseModel):
272                 return val.model_dump(mode="json")
273             return val
274 
275         def __getitem__(self, key: str) -> Any:
276             val = self.get(key)
277             if val is None and key not in type(self).model_fields:
278                 # Re-raise like a real dict for unknown top-level
279                 raise KeyError(key)
280             return val
281 
282 
283     # Convenience type alias for callers that want to annotate
284     Config = AppConfig
285 
```

10. user (2026-06-14T06:32:33.695Z)

```
82:    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
84:    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
96:    cue_flags: dict[str, bool] = Field(default_factory=dict)
99:    min_crossings: int = Field(default=0, ge=0)
102:    min_players: int = Field(default=0, ge=0)
112:    default_segmenter: str = "yolo_hybrid"
144:    @field_validator("default_segmenter")
```

11. assistant (2026-06-14T06:32:39.079Z)

Key findings so far. The default segmenter is `yolo_hybrid` (line 112), NOT fusion. `cue_flags` defaults to empty dict (line 96), `min_crossings`/`min_players` default 0 = OFF (lines 99, 102). Now let me verify the crux of the question: how cues enter the DECISION. The audit asks me to confirm cues enter ONLY as `feature_names` columns the learned scorer weights ? NOT a hard if/else.

I notice the `_gate_mask` function (lines 200-210) uses explicit `if variant.min_crossings > 0` / `if variant.min_players > 0` hard thresholds ? these ARE if/else branches. Let me examine this carefully and check the production fusion_hybrid segmenter and how features get persisted, plus check the served path.

12. user (2026-06-14T06:32:40.509Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
```

```
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                      frame_width: float, video_path: str,
78                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90              return stats
91
92          def _axis_estimate(self, points: List[Any]) -> tuple:
93              """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
```

```
it's
94              computed once per video, not once per candidate window (it depends only on
points)."""
95              key = id(points)
96              if self._axis_cache_key != key or self._axis_cache is None:
97                  self._axis_cache_key = key
98                  self._axis_cache = rally_gate.estimate_play_axis(points)
99              est = self._axis_cache
100             assert est is not None
101             return est
102
103         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                              frame_width: float, video_path: str,
105                              fcfg: Dict[str, Any]) -> Dict[str, float]:
106             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107             frame range and compute the scale/translation-invariant person features. Lazily
builds
108             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110             from backend.pipeline.detectors.person_detector import PersonDetector
111             if self._person is None:
112                 self._person = PersonDetector(self.config)
113
114             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119             fw = det["frame_width"] or frame_width
120             fh = det["frame_height"]
121             # If we can't trust the frame dims, every normalised feature would silently
collapse
122             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123             if fw <= 0 or fh <= 0:
124                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126             poly = fcfg.get("court_polygon")
127             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                               court_polygon=polygon, net=net)
132
133         def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                                 window_stats: List[Dict[str, Any]],
135                                 idx_cfg: Dict[str, Any]) -> List[int]:
136             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137             (0) keep every window ? identical to the substrate. Persisted candidates are
138             untouched (full superset)."""
139             fcfg = idx_cfg.get("fusion", {})
140             min_crossings = int(fcfg.get("min_crossings", 0))
141             min_players = int(fcfg.get("min_players", 0))
142             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144             keep: List[int] = []
145             for i, st in enumerate(window_stats):
146                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                     continue
148                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
```

```
min_players:
149                     continue
150                 keep.append(i)
151             return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155
```

13. user (2026-06-14T06:32:41.486Z)

```
26:        "default_segmenter": "gemini",
```

14. assistant (2026-06-14T06:32:50.141Z)

The shipped `config.json` sets `default_segmenter: "gemini"` (line 26), not fusion. Now I need to understand the critical distinction the audit is asking about. The COMPONENT is specifically the SCORING seam / no-special-casing. Let me think carefully about what "no special casing" means here.

There are TWO mechanisms in the variant for a signal entering the decision:
1. **Gate A/B** (`min_crossings`/`min_players`) ? these ARE hard if/else threshold branches (`_gate_mask`, lines 200-210; and `_select_for_handoff`, lines 146-149).
2. **Learned classifier** (`feature_names` ? LogisticRegression) ? the signal enters as a weighted feature COLUMN.

The audit's crux is whether cues enter ONLY as `feature_names` columns. Let me check the docs to understand the intended design ? whether gates are considered legacy/transitional or whether the person/shuttle cue is meant to flow through the learned scorer. Let me read the relevant docs and the fusion_features producer.

15. user (2026-06-14T06:32:51.086Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
```

```
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33          if polygon is None or len(polygon) < 3:
34              return False
35          x, y = point
36          inside = False
37          n = len(polygon)
38          j = n - 1
39          for i in range(n):
40              xi, yi = polygon[i]
41              xj, yj = polygon[j]
42              # Does the horizontal ray at y cross edge (i, j)?
43              if (yi > y) != (yj > y):
44                  x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                  if x < x_cross:
46                      inside = not inside
47              j = i
48          return inside
49
50
51      def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52          """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53          the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54          if frame_width <= 0 or frame_height <= 0:
55              return (0.0, 0.0)
56          x1, _y1, x2, y2 = box
57          return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60      def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
```

16. user (2026-06-14T06:32:53.049Z)

```
5:(SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.** Phases
below are
10:> [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels + `baseline.json`
come from
28:1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production
behavior.
30:3. Promote a variant to default **only on measured lift**, through a CI eval gate.
44:**decision**: "given those signals, where are the rallies" (windowing + gates + the
classifier).
75:### 3.3 No-regression gate + baselines (the promotion gate already exists)
86:`backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2):
`fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75),
`predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON)
(:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**
99:- `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)`
(:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`;
`FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40);
already does LOVO over Gemini silver labels.
135:Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B test,
and it needs
152:### 5.3 Promotion gate (no-regression, CI-enforced)
164:- **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no
silent default change.
```

```
173:## 7. What to build later (gap list ? thin, additive, owner-gated)
182:2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`,
`two_sided_motion`,
183:   `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`,
`direction_reversals`;
192:## 8. Phasing (owner-gated; ONE PR per slice, off `master`)
203:- **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
218:  "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test asserting
```

17. assistant (2026-06-14T06:32:58.337Z)

Let me look at the key parts of the harness doc that describe the relationship between gates and the learned classifier, particularly section 4 (the decision seam) and the distill_local FEATURE_NAMES which already includes net_crossings as a learned feature column.

18. user (2026-06-14T06:32:59.921Z)

```
36      discipline**, not a platform. §4 is the exact code surface so the next session does not
re-discover it.
37
38      ---
39
40      ## 2. The thesis ? separate expensive DETECTION from cheap DECISION
41
42      The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per
frame" (shuttle
43      WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python
part is the
44      **decision**: "given those signals, where are the rallies" (windowing + gates + the
classifier).
45
46      > **Run detection once per video, persist all per-cue signals + per-candidate features,
then re-score
47      > any number of variants OFFLINE ? deterministically, with no GPU and zero production
risk.**
48
49      This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do
exactly this on
50      stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence
substrate.
51      ? Most experiments are pure re-scoring of stored data; they never enter the served
pipeline.
52
53      ---
54
55      ## 3. What already exists (the audit ? do NOT rebuild)
56
57      Exact public surface, with `file:line`. **This table is the anti-duplicate-work
artifact** ? start here.
58
59      ### 3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
60      `backend/eval/metrics.py`
61      - `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
62      - `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1
IoU matching.
63      - `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1,
mean IoU, boundary errors; `Score.as_dict()`.
64      - `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).
65
66      ### 3.2 A/B + cross-validation primitives (the comparison engine already exists)
67      `backend/eval/calibration.py`
68      - `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5)
-> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
69      - `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113)
```

? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
70   - `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) ->
dict` (:72) ? within-video k-fold overfit guard.
71   - `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config,
float)` (:61).
72   - `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
73   - LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.
74
75   ### 3.3 No-regression gate + baselines (the promotion gate already exists)
76   `backend/eval/regression.py`
77   - `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None,
tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ?
score stored preds vs GT, flag if P **or** R drops > tolerance.
78   - `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
79   - `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)`
(:80) · `load_baselines(path) -> dict` (:68).
80   - `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R,
`tolerance`, `has_baseline`; `as_dict()`.
81   - Default baseline file: `eval_baselines/regression_baseline.json` (:33).
82
83   `run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1
regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations
--stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-
missing-baseline --json`.
84
85   ### 3.4 Pinnable model artifact
86   `backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2):
`fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75),
`predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON)
(:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**
87
88   ### 3.5 Offline re-scoring substrate (persist once, re-score forever)
89   `backend/infrastructure/database.py`
90   - `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None,
confidence=None, stats=None, detection_params=None) -> Optional[int]` (:240) ?
`stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`,
`detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
91   - `query_candidate_segments(video_id, detector=None, only_unlabeled=False) ->
List[dict]` (:277) ? returns rows with deserialized `stats`.
92
93   ### 3.6 Label assignment + feature glue
94   `backend/eval/training.py`
95   - `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]`
(:50) ? y-labels by IoU match to golden (same matcher as the evaluator).

19. user (2026-06-14T06:33:01.083Z)

```
11:  active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
40:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
116:            model = LogisticRegression().fit(X, y, FEATURE_NAMES)
135:    final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
137:    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
```

20. assistant (2026-06-14T06:33:07.877Z)

This is the key evidence. `distill_local.py` line 40 shows `net_crossings` is already a learned FEATURE COLUMN: `FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]`, fed to `LogisticRegression().fit(X, y, FEATURE_NAMES)` (line 116). So the shuttle cue (`net_crossings`) enters the decision as a weighted feature column.

Now let me verify the crux precisely. The audit claim: "cues enter ONLY as extra `feature_names` columns the learned scorer weights ? adding/removing a cue is a feature-list change, never an if/else in the decision path."

There's a nuance: the `Variant` ALSO supports hard-threshold gates (`min_crossings`/`min_players` via `_gate_mask`). These ARE if/else branches. I need to assess whether the design intends these as the cue-entry path or as a separate, legacy/transitional knob. Let me check the tests to understand the canonical usage and the MULTI_SIGNAL_FUSION_PLAN positioning of gates vs the learned scorer.

21. user (2026-06-14T06:33:08.873Z)

```
1       """Tests for the experiment harness (backend.eval.experiment).
2
3       Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the
4       **no-regression invariant** (`test_disabled_cues_byte_identical_to_control`): a variant
5       that merely carries a disabled cue must predict exactly what control predicts. That is
the
6       guarantee that lets new cues be merged freely (`EXPERIMENT_HARNESS.md` §6).
7       """
8       import json
9
10      import pytest
11
12      from backend.eval.classifier import LogisticRegression
13      from backend.eval.experiment import (
14          CONTROL,
15          ExperimentError,
16          Variant,
17          VideoCandidates,
18          compare,
19          compare_unlabeled,
20          evaluate_variant,
21          load_video_candidates,
22          predict,
23          record_experiment,
24      )
25      from backend.infrastructure.database import Database
26
27
28      # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
29      # not overlap any golden rally, so it is a negative by construction.
30      def _held_shuttle_video(name="v", gts=None):
31          return VideoCandidates(
32              name=name,
33              intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
34              features={"net_crossings": [4.0, 3.0, 0.0]},
35              gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
36          )
37
38
39      # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
40      # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
41      def _separable_videos():
42          vids = []
43          for k in range(3):
44              base = 100.0 * k
45              vids.append(VideoCandidates(
46                  name=f"sep{k}",
47                  intervals=[(base + 0, base + 10), (base + 20, base + 30),
48                             (base + 40, base + 41), (base + 50, base + 51)],
49                  features={"x": [5.0, 5.0, 0.0, 0.0]},
50                  gts=[(base + 0, base + 10), (base + 20, base + 30)],
51              ))
52          return vids
53
54
55      # ---------------------------------------------------------------------- Variant
56
```

```
    def test_variant_roundtrip_and_hash_stability():
        v = Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
        assert Variant.from_dict(v.to_dict()) == v
        assert v.spec_hash() == v.spec_hash()                      # stable
        assert CONTROL.spec_hash() == Variant(name="control").spec_hash()
        assert v.spec_hash() != CONTROL.spec_hash()              # different recipe ?
different hash


    def test_from_dict_ignores_unknown_keys():
        v = Variant.from_dict({"name": "x", "min_crossings": 1, "bogus": 99})
        assert v.name == "x" and v.min_crossings == 1


    def test_is_learned():
        assert not CONTROL.is_learned()
        assert Variant(name="r", feature_names=["x"]).is_learned()
        assert Variant(name="p", classifier_model="model.json").is_learned()


    # ------------------------------------------------------ the no-regression invariant

    def test_disabled_cues_byte_identical_to_control():
        """A variant carrying only DISABLED cues must equal control, window-for-window."""
        vc = _held_shuttle_video()
        disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
                           min_crossings=0, min_players=0)
        assert predict(disabled, vc) == predict(CONTROL, vc)


    def test_control_is_merged_candidates():
        vc = VideoCandidates(name="v", intervals=[(0.0, 10.0), (10.5, 12.0), (40.0, 50.0)])
        # merge_gap=1.0 stitches the 0.5s gap between the first two; the 28s gap stays
split.
        assert predict(CONTROL, vc) == [(0.0, 12.0), (40.0, 50.0)]


    # ------------------------------------------------------------------ decision gates

    def test_gate_a_drops_low_crossing_window():
        vc = _held_shuttle_video()
        gate_a = Variant(name="gateA", min_crossings=2)
        preds = predict(gate_a, vc)
        assert (12.0, 13.0) not in preds                         # held-shuttle fire
removed
        assert preds == [(0.0, 10.0), (20.0, 30.0)]
        assert preds != predict(CONTROL, vc)                      # the gate actually
changed output


    def test_gate_b_filters_on_players():
        vc = VideoCandidates(
            name="v", intervals=[(0.0, 10.0), (20.0, 30.0)],
            features={"players_in_court": [2.0, 1.0]})
        preds = predict(Variant(name="gateB", min_players=2), vc)
        assert preds == [(0.0, 10.0)]                             # the 1-player window is
dropped


    def test_gate_on_absent_feature_defaults_to_zero(caplog):
        vc = VideoCandidates(name="v", intervals=[(0.0, 10.0)])    # no net_crossings
persisted
        preds = predict(Variant(name="gateA", min_crossings=2), vc)
        assert preds == []                                        # absent feature ? 0 ?
fails gate
```

```
115
116
117     # ------------------------------------------------------------ evaluate_variant
118
119     def test_evaluate_static_variant_scores_vs_gts():
120         res = evaluate_variant([_held_shuttle_video()], CONTROL, iou=0.5)
121         pv = res["per_video"][0]
122         # control keeps the false fire: 2 TP, 1 FP ? precision 2/3, recall 1.0
123         assert pv["recall"] == pytest.approx(1.0)
124         assert pv["precision"] == pytest.approx(2 / 3)
125         assert res["aggregate"]["recall"] == pytest.approx(1.0)
126         assert not res["honest_lovo"]
127
128
129     def test_evaluate_learned_lovo_recovers_separable_signal():
130         variant = Variant(name="learned", feature_names=["x"], threshold=0.5)
131         res = evaluate_variant(_separable_videos(), variant, iou=0.5, honest=True)
132         assert res["honest_lovo"] is True
133         assert res["aggregate"]["f1"] == pytest.approx(1.0)        # held-out F1 perfect on
separable data
134
135
136     def test_evaluate_learned_lovo_needs_two_videos():
137         variant = Variant(name="learned", feature_names=["x"])
138         with pytest.raises(ExperimentError):
139             evaluate_variant(_separable_videos()[:1], variant, honest=True)
140
141
142     def test_evaluate_requires_labels():
143         with pytest.raises(ExperimentError):
144             evaluate_variant([VideoCandidates(name="v", intervals=[(0.0, 1.0)], gts=None)],
CONTROL)
145
146
147     # ------------------------------------------------------------------------ compare
148
149     def test_compare_gate_a_improves_precision():
150         videos = [_held_shuttle_video()]
151         res = compare(videos, CONTROL, Variant(name="gateA", min_crossings=2), iou=0.5)
152         assert res["variant_b"]["aggregate"]["precision"] == pytest.approx(1.0)
153         assert res["delta"]["precision"] > 0                       # B (gate) beats A
(control) on precision
154         assert res["delta"]["f1"] > 0
155         assert res["label_set_hash"]                               # fingerprint present
156         assert res["num_videos"] == 1
157         assert len(res["per_video_delta"]) == 1
158
159
160     def test_compare_needs_a_video():
161         with pytest.raises(ExperimentError):
162             compare([], CONTROL, CONTROL)
163
164
165     # ------------------------------------------------------------------ compare_unlabeled
166
167     def test_compare_unlabeled_partitions_agreement():
168         vc = _held_shuttle_video(gts=None)                         # new footage, no labels
169         res = compare_unlabeled(vc, CONTROL, Variant(name="gateA", min_crossings=2))
170         assert res["num_a"] == 3 and res["num_b"] == 2
171         assert res["num_agreed"] == 2
172         assert res["a_only"] == [(12.0, 13.0)]                     # A fired, B suppressed
173         assert res["b_only"] == []                                 # B is a subset of A here
174         assert res["mean_agree_iou"] == pytest.approx(1.0)
175
176
```

```
177     def test_compare_unlabeled_learned_variant_needs_pinned_model():
178         vc = VideoCandidates(name="new", intervals=[(0.0, 10.0)], features={"x": [5.0]})
179         learned_no_model = Variant(name="learned", feature_names=["x"])
180         with pytest.raises(ExperimentError):
181             compare_unlabeled(vc, CONTROL, learned_no_model)
182 
183 
184     def test_compare_unlabeled_with_pinned_model(tmp_path):
185         # Train + persist a model, then use it as a pinned variant on an unlabeled video.
186         model = LogisticRegression().fit([[5.0], [5.0], [0.0], [0.0]], [1, 1, 0, 0], ["x"])
187         model_path = str(tmp_path / "m.json")
188         model.save(model_path)
189         vc = VideoCandidates(name="new", intervals=[(0.0, 10.0), (40.0, 41.0)],
190                              features={"x": [5.0, 0.0]})
191         pinned = Variant(name="learned", feature_names=["x"], classifier_model=model_path)
192         res = compare_unlabeled(vc, CONTROL, pinned)
193         assert res["num_a"] == 2                                    # control keeps both
194         assert res["num_b"] == 1                                    # model drops the x=0
window
195         assert res["a_only"] == [(40.0, 41.0)]
196 
197 
198     # ------------------------------------------------------------------- record_experiment
199 
200     def test_record_experiment_appends(tmp_path):
201         path = str(tmp_path / "leaderboard.json")
202         res = compare([_held_shuttle_video()], CONTROL, Variant(name="gateA",
min_crossings=2))
203         row1 = record_experiment(res, path, timestamp="2026-06-13T00:00:00Z")
204         row2 = record_experiment(res, path, timestamp="2026-06-13T01:00:00Z")
205         assert row1["variant_b"]["name"] == "gateA"
206         assert row1["timestamp"] != row2["timestamp"]
207         with open(path, encoding="utf-8") as f:
208             entries = json.load(f)
209         assert len(entries) == 2                                    # appended, not
overwritten
210         assert entries[0]["delta"]["precision"] > 0
211 
212 
213     # ------------------------------------------------------------------
load_video_candidates
214 
215     def test_load_video_candidates_from_db(tmp_path):
216         db = Database(str(tmp_path / "t.db"))
217         db.add_video("vid1", "/x.mp4", "ok", [])
218         db.add_candidate_segment("vid1", "wasb_hybrid", 0.0, 10.0,
219                                  stats={"net_crossings": 4, "peak_velocity": 0.3})
220         db.add_candidate_segment("vid1", "wasb_hybrid", 12.0, 13.0, stats={"net_crossings":
0})
221 
222         vc = load_video_candidates(db, "vid1", detector="wasb_hybrid", gts=[(0.0, 10.0)],
223                                    feature_names=["net_crossings", "duration"])
224         assert vc.intervals == [(0.0, 10.0), (12.0, 13.0)]
225         assert vc.features["net_crossings"] == [4.0, 0.0]
226         assert vc.features["duration"] == [10.0, 1.0]               # derived from bounds
227         assert vc.gts == [(0.0, 10.0)]
228 
```

22. assistant (2026-06-14T06:33:17.719Z)

Now I have a full picture. Let me assess the nuance carefully. The audit's crux question is: do cues enter ONLY as `feature_names` columns the learned scorer weights, never as if/else?

There IS a complication: the `Variant` exposes BOTH paths:
1. **Learned scorer path** ? `feature_names` ? `LogisticRegression` weighted columns (lines

227-238). This is the clean "no special casing" path: `net_crossings` is a weighted column (confirmed in distill_local FEATURE_NAMES).
2. **Gate path** ? `min_crossings`/`min_players` hard thresholds in `_gate_mask` (lines 200-210) and `_select_for_handoff` (lines 146-149). These ARE hard-coded if/else threshold branches.

So the harness is NOT *purely* "no if/else" ? it intentionally retains hard-threshold gates (Gate A / Gate B). I need to characterize this honestly. The audit asks me to "confirm cues enter ONLY as feature_names columns ... NEVER an if/else in the decision path." Strictly, that is FALSE ? the gates are an alternative if/else entry path that exists by design. However, the learned-scorer seam itself is correctly built and the same cue (net_crossings) flows as a weighted column.

Let me verify the MULTI_SIGNAL_FUSION_PLAN positioning of gates relative to the learned scorer, to determine whether gates are framed as a transitional/cheap-win OR as the production decision path.

23. user (2026-06-14T06:33:20.219Z)

5:behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a separate PR.
12:> The vendor labels that gate the learned path come from GOLDEN_SET_VENDORING.md
16:> below lands flag-gated, default OFF, promoted only on measured LOVO lift).
44:- **`backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate, explicitly
45: labelled in its own docstring as *"Gate A in the over-fire fix spectrum (**B = player-stance state
46: machine, future**)."* The person cue in this plan **is that named-but-unbuilt Gate B.**
51:points. This is the *exact* failure `rally_gate.py` was written to kill:
58:runtime path.** `rally_gate.apply_gate(..., min_crossings=2)` is called only from the **eval drivers**
62:Gate A into production.** Everything else (the person cue / Gate B, the learned fusion) is robustness
75:- **Recall** (via the learned classifier) ? features that help bridge shuttle occlusion/blur gaps.
82:activity and/or per-window features); **fusion happens in *our* layer, never inside a frozen model.**
88:[frames ?? pose/stance (ours; parked #3) ?? stance ????????   feature + decision fusion
95:- **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-window stats.
103:### 3.1 Decision-level gate (safe baseline, ships first, no labels needed)
109:The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate can't cause a
110:recall cliff. This composes with the **existing net-crossing Gate A**:
113:keep window ? net_crossings ? 2   (Gate A ? EXISTS in rally_gate.py, eval-only today)
114:              ? (?2 in-court players ? recent track)   (Gate B ? the person cue, NEW)
117:Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-occupancy evidence
121:### 3.2 Feature-level (primary, the learned path)
123:The cues emit a **small set of aggregated, scale/translation-invariant per-window features ? NOT raw
127:**What already feeds the classifier** (`distill_local.py:40`, `FEATURE_NAMES`):
128:`duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note **`net_crossings` is
129:already a feature** (computed via `rally_gate.gate_windows`).
131:**Person features to add (~4, deliberately few):**
133:| Feature | Meaning | Rally signal |
137:| `mean_player_motion` | aggregate normalized player velocity | rally = sustained; resting/standing = low |
141:These join the shuttle features in the candidate `stats` JSON
142:(`database.py::add_candidate_segment`) and the feature vector of the **existing distilled classifier ?
143:the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004), NOT a new net.**

146:**What the model learns:** the **weights** on those few features ? e.g. down-weight a shuttle-motion
151:- Keep to **~4 person features** ? more features on a still-small labeled set overfit; that's exactly
159: training labels. Until the labeled set is larger, treat the learned weights as
160: **calibration / direction-check, not "a fully general model"** ? the decision-level gate (§3.1) is
163:> *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
175:| `net_crossings` | ? 1 (usually several) | ~0 (held) / ~1 (single feed) | **EXISTS** (`rally_gate.py`); eval-only ? wire to prod |
180:So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation feature from the
193: this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
202:- **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never the sole
209:**Box-first.** Occupancy + count + two-sided motion + shuttle-colocation is enough for both the gate

[Showing results with pagination = limit: 40]

24. assistant (2026-06-14T06:33:29.770Z)

This clarifies the design intent fully. The plan explicitly has TWO complementary fusion levels:
- **§3.1 Decision-level gate** (Gate A/B) ? a "safe baseline, ships first, no labels needed" hard threshold. This is intentionally an if/else (`net_crossings ? 2`).
- **§3.2 Feature-level (primary, the learned path)** ? cues emit feature columns that the LogisticRegression weights. `net_crossings` is "already a feature" (line 128). Person features "join the shuttle features ... the feature vector of the existing distilled classifier."

So the answer is nuanced and I must be honest. The SCORING SEAM (the learned path) is correctly built: the same cue (`net_crossings`) enters as a weighted feature column in `feature_names` ? LogisticRegression. Adding the person cue is a feature-list change (add `players_in_court` etc. to `feature_names`). That seam has NO special-casing.

BUT the harness ALSO retains hard-threshold gates (`min_crossings`/`min_players` if/else) as a deliberate, separate, default-OFF decision-level path ? both in the offline `Variant` (`_gate_mask`) and in the served `fusion_hybrid._select_for_handoff`. These are genuine if/else branches in the decision path. They are by-design (the "cheapest win" / Gate A) and gated default-OFF, not a smuggled special-case of a cue into the learned scorer.

Let me confirm the n=6 / retained claim ? that nothing was deleted after the non-significant ?F1. Let me check the recent commits and current-state memory.

25. user (2026-06-14T06:33:31.466Z)

2e5892e feat(eval): fix the learned variant ? train-fold threshold tuning + calibration (C2) (#130)
f680b8d feat(eval): wire C1+C4 honest reporting into experiment.compare() (additive) (#129)
ecee786 feat(eval): experiment harness ? A/B + SxS variant layer (no-regression by design) (#114)
df9cf80 ci: add mypy lane (lenient green baseline + explicit quarantine list) (#93)
595a5a1 ci: add ruff lint lane + fix all violations; remove never-compiled flyer stub (#92)
8615534 refactor: adopt approved layered directory structure
0c6f2c1 feat(eval): distilled local rally classifier + recover calibrate_local (#31)
8a5844a feat(phase4): dependency-free numpy logistic-regression classifier

26. user (2026-06-14T06:33:33.854Z)

12:**Checkpoint:** 2026-06-14 (FUSION P3/P4 MEASURED + extraction COMPLETE + telemetry #134 merged + Cooler Boost). **THE NUMBERS
13:(honest LOVO n=6, IoU?0.5, C1 bootstrap-CI + sign-test):** shuttle-only F1=**0.307**; **+person (P3) ?F1=?0.021** (?6.7%, CI
15:counts spectators; per-video wash: testlarge +0.40 / gbaaddy ?0.22). **+person+audio (P4 incremental) ?F1=+0.079** (+27.7%, F1

17:+0.27; endpoint 0.366 > shuttle-only 0.307) but n=6 can't confirm. **VERDICT: neither promotable (both CIs span 0); audio is
20:result into `docs/QUALITY_ITERATIONS.md` (in flight); **wire `backend/utils/run_telemetry.py` (#134, MERGED) into fusion_golden
21:+ the ps1 wrapper** (the deferred follow-up). **Extraction infra** (all working): detached Scheduled Task `FusionGoldenExtract`
51:validated on n=6 golden ? **mean held-out F1 0.307?0.423 (+0.116), testlarge 0.000?0.300 (un-zeroed), now BEATS CONTROL
53:QUALITY_ITERATIONS ? measured win dF1 +0.074, 6/6, p=0.031; recommended `indexing.fusion.min_crossings=2`; **live served-default
54:flip stays OWNER-GATED** until a real fusion-path run confirms ? fusion P2 is unverified on real video) **+ new
63:isolated worktrees off master (siblings concurrent). ?? NEXT (owner-gated): real fusion-path run to confirm Gate-A served-side
68:**first real end-to-end A/B on the n=6 golden corpus, 100% OFFLINE** (re-scored persisted WASB trajectories via
70:three ways: **Gate-A (net_crossings?2) is a MEASURED WIN ? ?F1 +0.074, 6/6 matches, sign-test p=0.031, CI
77:`honest` block = per-metric bootstrap CIs + paired ?F1 CI + exact sign test + F1@k + segment_count_ratio; `record_experiment`
78:carries honest ?F1; existing fields byte-unchanged, `honest_stats=False` = legacy shape). Reuses #124 helpers; complements
79:sibling **#127** which wired the same C1 into `fusion_compare` (no dup). 19 existing experiment tests pass + 3 new;
85:**Checkpoint:** 2026-06-14 (FUSION P3/P4 MEASUREMENT BUILT + GPU EXTRACTION RUNNING ? master `d7b1f51`; suite 514 green).
86:The "modeling" track for the person-cue (P3) + audio (P4) fusion lift ? all default-OFF, no-special-casing, harness-measured.
87:**Merged this run (mine): #121** fusion P3 (`backend/eval/fusion_golden.py` offline person-feature EXTRACTOR + `fusion_compare.py`
88:LOVO driver ? reuses build_candidates/PersonDetector/fusion_features/experiment.compare, NO new scoring); **#122** P4
89:(`backend/pipeline/detectors/audio_features.py` + `backend/eval/fusion_audio.py` incremental audio cue); **#123** progress-logging
91:pathology ? ~28s/window ? one sequential decode); **#126** `--smallest-first`; **#127** honest ?F1 = bootstrap CI + paired
92:sign test on per-video held-out F1 (wires #124's `eval_stats.paired_delta_ci`; "never a bare mean"). Concurrent sessions
94:**? GPU EXTRACTION via DETACHED SCHEDULED TASK `FusionGoldenExtract`** (runs `scratch/extract_until_done.ps1` ? Task Scheduler
96:6/6; clean UTF-8 logs to `output/fusion_golden.log` + `scratch/extract_wrapper.log`). Loops `python -u -m backend.eval.fusion_golden
99:kushagra?gbaaddy?adarsh(last, 84k frames). **MONITOR:** JSON count + `tail output/fusion_golden.log` + `nvidia-smi`;
100:`Get-ScheduledTask FusionGoldenExtract|Select State`. **RESTART (resumes, skips done):** `Start-ScheduledTask -TaskName
101:FusionGoldenExtract`. **STOP:** `Stop-ScheduledTask -TaskName FusionGoldenExtract` + `Get-Process python|Stop-Process -Force`.
115:the NUMBERS:** (1) `python -m backend.eval.fusion_compare --features-dir output --record` ? honest P3 LOVO ?F1 (shuttle-only
116:vs +person) w/ CI+sign-test; (2) `python -m backend.eval.fusion_audio --augment --compare --record` ? P4 incremental audio
117:?F1; (3) log both to `docs/QUALITY_ITERATIONS.md` I3 (HONEST measured numbers only ? [[paper-coauthor-aim]]). Honest caveat:
121:**STORAGE (this session, owner-approved "delete cache + compact WSL"):** deleted the stale **93GB GX010125 frame cache**
130:reframed the analyzers/reconcilers design into a **multi-scope signal-fusion framework** (frame detectors / window analyzers /
134:reconciler-as-refinement = Temporal Action Segmentation post-proc; "signal not verdict" = late/score-level fusion + stacked
140:`bootstrap_ci`/`paired_sign_test`/`paired_delta_ci` ? never a bare LOVO mean at n?6; C3 `out_of_fold_predictions`/`grouped_folds`
144:recurring shared-checkout branch-flips (siblings merged #117 fusion-P2, #118, #120, #123 concurrently; **#116 closed ? its content
151:on verified-green CI per [[working-agreement-merging]] (squash; remote+local branch

deleted); was branch `feat/runlog-readability`
168:question ? left untouched** (must KEEP `feat/khelacharya-shot-intelligence` #11 =
deliberately-retained sibling layer per
176:signal-fusion framework**: producers emit SIGNALS at **3 temporal scopes** ? frame detectors
(`RallyCue`) / window analyzers
181:**frame-scope detector** (a MULTI_SIGNAL_FUSION_PLAN `RallyCue`, NOT a window analyzer) ?
derive-from-existing-signals first
185:ablations + >1-sport still owed, n=6 = direction-check only ? the honest backbone the paper
aim needs. Diff = `2-seam?framework`

[Showing results with pagination = limit: 40]

27. assistant (2026-06-14T06:33:40.132Z)

This confirms the "retained" claim decisively. The current-state memory (lines 12-17) shows the measured results: P3 person ?F1=?0.021 and P4 audio ?F1=+0.079, both with CIs spanning 0 at n=6 ? "neither promotable." Yet the code (producers + feature math + the experiment seam) is all retained, default-OFF, re-measurable. Nothing was deleted.

Let me do one final check on the served path to confirm the learned scorer does NOT run in production by default (the production segmenter `yolo_hybrid`/`gemini` does not invoke the experiment harness's LogisticRegression scoring). Let me verify there's no production import of `experiment.predict` or the classifier in the served handoff path.

28. user (2026-06-14T06:33:42.254Z)

No matches found

29. assistant (2026-06-14T06:33:50.399Z)

Confirmed: no production (non-eval) code imports the experiment harness or the LogisticRegression scorer. The learned scorer lives entirely in the offline eval layer; the served pipeline (`yolo_hybrid`/`gemini`/`fusion_hybrid`) never invokes it by default.

I now have all evidence. Let me synthesize my skeptical assessment. The key honest nuance: the SCORING SEAM (the learned-classifier path) is built exactly as the audit describes ? cues enter as weighted `feature_names` columns, adding a cue is a feature-list change. BUT the `Variant`/`fusion_hybrid` ALSO deliberately retains hard-threshold Gate A/B (`min_crossings`/`min_players`) if/else branches as a *separate, default-OFF, label-free* decision-level path. So "cues enter ONLY as feature columns, NEVER if/else" is not strictly true ? there is a by-design if/else gate path alongside the learned seam. I'll flag this precisely as a concern.

30. user (2026-06-14T06:34:21.008Z)

Structured output provided successfully

31. assistant (2026-06-14T06:34:32.662Z)

I've completed the audit of the SCORING seam / no-special-casing component and returned the structured findings.

Summary of the verdict:

- **present: true** ? The learned scorer seam exists: `Variant.feature_names` + a generic numpy `LogisticRegression` in `backend/eval/classifier.py`, wired through `predict()` and `compare()`.
- **default_off: true** ? Production default is `yolo_hybrid` (config model) / `gemini` (shipped config.json), never the fusion/learned path; `cue_flags={}`, `min_crossings=0`, `min_players=0`. No non-eval code imports the harness or the classifier.
- **no_special_casing: true** ? In the learned path (`predict`, lines 227-238) a cue enters ONLY as a name in `feature_names` ? a column in `_feature_matrix` ? a weighted column in LogisticRegression. The scorer has zero cue-specific logic. `distill_local.py:40` proves `net_crossings` already rides as a weighted column. Adding the person cue is a feature-list change.

- **retained: true** ? Despite the non-significant n=6 ?F1 (P3 person ?0.021, P4 audio +0.079, both CIs span 0, "neither promotable"), all producers + feature math + the seam are intact and re-measurable. Nothing deleted.

The load-bearing caveat I flagged: the literal claim "cues enter ONLY as feature columns, NEVER an if/else" is **not strictly true**. The harness deliberately retains a *second*, default-OFF, label-free decision path ? the hard-threshold Gate A/B (`_gate_mask` at experiment.py:200-210; the served `_select_for_handoff` at fusion_hybrid.py:144-151), which are genuine `if net_crossings < min_crossings` branches. This is by design (MULTI_SIGNAL_FUSION_PLAN §3.1 decision-gate vs §3.2 learned path), it is default-OFF, and it is NOT a cue smuggled into the learned scorer ? so it doesn't violate the spirit of "no special-casing." But it does mean the learned-column seam is not the *only* way a cue reaches the decision, and notably the currently-recommended win (Gate-A `min_crossings=2`, p=0.031) is the if/else gate, not the learned column.